ESCOLA Máximu's
INFANTIL • FUNDAMENTAL • MÉDIO • TÉCNICO

VERSÃO SIMPLES / COM TODAS AS RESPOSTAS

Java + MySQL
no Eclipse

Uma conexão curta.
Um arquivo para cada exercício.
Dez respostas completas para estudar.

01 Instale e prepare o banco

02 Execute a primeira conexão

03 Estude e rode os dez exercícios

Prof. Bruno Michel Pera

JDK 17+ • MySQL local • Escola Máximu's

CÓDIGOS E GABARITO: github.com/perabru/java-mysql

01 / DOWNLOADS

O que você precisa

Clique nos links oficiais para baixar as ferramentas.

JDK 21 LTS
https://adoptium.net/temurin/releases/?version=21

Compila e executa os programas. O código aceita Java 17 ou superior.

Eclipse IDE for Java Developers
https://www.eclipse.org/downloads/packages/

Editor para escrever e executar Java. Mantenha o runtime exigido pelo instalador da IDE.

MySQL Community Server 8.4 LTS
https://dev.mysql.com/downloads/mysql/8.4.html

Servidor que armazena os dados. Use a porta 3306 na configuração local.

MySQL Workbench
https://dev.mysql.com/downloads/workbench/

Ferramenta visual para executar SQL. O Workbench não substitui o servidor.

Connector/J - driver JDBC
https://dev.mysql.com/downloads/connector/j/

No caminho Maven, o driver já é baixado automaticamente.

Connector/J 9.7.0 - instalação manual
https://downloads.mysql.com/archives/c-j/

Escolha a versão 9.7.0, Platform Independent, ZIP. Extraia o arquivo .jar.

> **Confira o JDK:** abra o terminal e execute java -version e javac -version. Ambos devem mostrar Java 17 ou superior. No MySQL, mantenha a autenticação e TLS padrão.

02 / BANCO

Prepare os dados

No Workbench, entre como root com a senha criada na instalação.

01 Crie banco e tabela

Abra sql/01_banco.sql com File → Open SQL Script e execute o script inteiro pelo botão de raio.

```
-- Execute como administrador do MySQL. Nao apaga registros existentes.
CREATE DATABASE IF NOT EXISTS escola_maximus
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE escola_maximus;
CREATE TABLE IF NOT EXISTS alunos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE
);
```

02 Crie o usuário da aula

Execute sql/02_usuario_local.sql como administrador. Ele concede apenas acesso aos dados deste banco.

```
CREATE USER IF NOT EXISTS 'aluno_java'@'localhost'
  IDENTIFIED BY 'MaximusLocal_2026!';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON escola_maximus.* TO 'aluno_java'@'localhost';
```

O banco se chama **escola_maximus**. A tabela **alunos** tem id automático, nome e email único. Os scripts não apagam os alunos existentes.

> **Senha pública da aula:** use esta configuração apenas no próprio computador. Se o usuário já existia, o script preserva a senha antiga. Nesse caso, informe a senha correta na classe Conexao.

03 / ECLIPSE

Abra e execute

O caminho mais curto usa o projeto Maven pronto.

Baixar o projeto completo em ZIP
https://github.com/perabru/java-mysql/archive/refs/heads/main.zip

01 Extraia e importe

File → Import → Maven → Existing Maven Projects. Selecione a pasta que contém pom.xml e clique em Finish.

02 Aguarde o driver

O primeiro download precisa de internet. Se necessário, botão direito no projeto → Maven → Update Project.

03 Abra a conexão

Em src/main/java → (default package), abra Conexao.java e confira URL, usuário e senha.

04 Execute o primeiro programa

Abra Exercicio01.java, clique com o botão direito no editor → Run As → Java Application. A resposta aparece no Console.

Se a aula pedir Java Project sem Maven

Crie File → New → Java Project, sem module-info.java. Copie os arquivos .java para src, sem criar pacote. Crie lib, copie o JAR extraído do Connector/J e adicione em Build Path → Configure Build Path → Libraries → **Classpath → Add JARs** → Apply and Close.

> **Um arquivo por exercício:** Conexao.java é compartilhado. Execute Exercicio01, Exercicio02 e assim por diante. Nas classes 08 e 10, cada execução faz uma operação; rode novamente para escolher outra. O exercício 09 mantém o menu aberto.

No terminal, com Maven instalado: **mvn compile** e **mvn exec:java -Dexec.mainClass=Exercicio01**. Troque o número para rodar outra resposta.

04 / CONEXÃO

Só o essencial

Arquivo completo: Conexao.java

```
import java.sql.*;

public class Conexao {
    public static Connection conectar() throws SQLException {
        String url = "jdbc:mysql://localhost:3306/escola_maximus"
            + "?sslMode=REQUIRED";
        String usuario = "aluno_java";
        String senha = "MaximusLocal_2026!"; // Senha publica da aula.
        return DriverManager.getConnection(url, usuario, senha);
    }
}
```

Entenda as três variáveis

url: aponta para localhost, porta 3306 e banco escola_maximus.
usuario: aluno_java, criado no SQL.
senha: a senha local configurada para esse usuário.

DriverManager.getConnection abre a comunicação. **Connection** representa a conexão. O import java.sql.* importa os tipos JDBC usados no exemplo.

Os comandos que aparecem nas respostas

prepareStatement: prepara o SQL.
setString / setInt: preenchem os valores de ?.
executeUpdate: insere, altera ou exclui.
executeQuery: consulta e devolve um ResultSet.
rs.next(): avança para a próxima linha.
try (...): fecha os recursos automaticamente.

> **Simplicidade didática:** os primeiros exemplos usam throws SQLException para mostrar erros do banco. O exercício 09 ensina catch e repetição. A senha é pública para a aula; não publique credenciais reais. Para servidores remotos, configure VERIFY_IDENTITY e certificado confiável.

RESPOSTA 01

Primeira conexão

Arquivo completo: Exercicio01.java

Enunciado: Instale o ambiente e conecte ao banco.

```
import java.sql.*;

public class Exercicio01 {
    public static void main(String[] args) throws SQLException {
        try (Connection c = Conexao.conectar()) {
            System.out.println("Conectado: " + c.getCatalog());
        }
    }
}
```

Como conferir: Conectado: escola_maximus. Host é o computador do banco; porta é o ponto de acesso (3306); escola_maximus é o nome do banco.

RESPOSTA 02

Três cadastros

Arquivo completo: Exercicio02.java

Enunciado: Cadastre três alunos e confira os registros no Java e no Workbench.

```
import java.sql.*;

public class Exercicio02 {
    public static void main(String[] args) throws SQLException {
        String[] nomes = {"Ana", "Bruno", "Carla"};
        String[] emails = {"ana@example.com", "bruno@example.com",
                           "carla@example.com"};
        String sql = "INSERT INTO alunos (nome, email) VALUES (?, ?)";
        try (Connection c = Conexao.conectar();
             PreparedStatement ps = c.prepareStatement(sql)) {
            for (int i = 0; i < nomes.length; i++) {
                ps.setString(1, nomes[i]);
                ps.setString(2, emails[i]);
                try {
                    ps.executeUpdate();
                    System.out.println("Cadastrado: " + nomes[i]);
                } catch (SQLException e) {
                    if (e.getErrorCode() != 1062) throw e;
                    System.out.println("E-mail ja cadastrado: " + emails[i]);
                }
            }
        }
    }
}
```

Como conferir: Execute 02 e depois 04 com uma linha vazia para listar todos. No Workbench, SELECT * FROM alunos mostra IDs distintos. Repetir 02 informa os e-mails já cadastrados sem duplicar registros.

RESPOSTA 03

Buscar por ID

Arquivo completo: Exercicio03.java

Enunciado: Mostre nome/e-mail de um aluno ou informe que o ID não existe.

```
import java.sql.*;
import java.util.Scanner;

public class Exercicio03 {
    public static void main(String[] args) throws SQLException {
        try (Scanner teclado = new Scanner(System.in)) {
            System.out.print("ID do aluno: ");
            int id = Integer.parseInt(teclado.nextLine());
            String sql = "SELECT nome, email FROM alunos WHERE id = ?";
            try (Connection c = Conexao.conectar();
                 PreparedStatement ps = c.prepareStatement(sql)) {
                ps.setInt(1, id);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        System.out.println(rs.getString("nome"));
                        System.out.println(rs.getString("email"));
                    } else {
                        System.out.println("Aluno nao encontrado.");
                    }
                }
            }
        }
    }
}
```

Como conferir: Digite o ID obtido no exercício 2. Depois teste 999999 para conferir a mensagem de aluno não encontrado.

RESPOSTA 04

Pesquisar por nome

Arquivo completo: Exercicio04.java

Enunciado: Busque parte do nome com LIKE e ordene por nome.

```
import java.sql.*;
import java.util.Scanner;

public class Exercicio04 {
    public static void main(String[] args) throws SQLException {
        try (Scanner teclado = new Scanner(System.in)) {
            System.out.print("Parte do nome: ");
            String nome = teclado.nextLine();
            String sql = "SELECT id, nome FROM alunos "
                + "WHERE nome LIKE ? ORDER BY nome";
            try (Connection c = Conexao.conectar();
                 PreparedStatement ps = c.prepareStatement(sql)) {
                ps.setString(1, "%" + nome + "%");
                try (ResultSet rs = ps.executeQuery()) {
                    boolean encontrou = false;
                    while (rs.next()) {
                        encontrou = true;
                        System.out.println(rs.getInt("id") + " - "
                            + rs.getString("nome"));
                    }
                    if (!encontrou) System.out.println("Nenhum aluno.");
                }
            }
        }
    }
}
```

Como conferir: Digite a para vários nomes, zzz para nenhum e deixe vazio para listar todos. O termo é passado ao parâmetro, não concatenado ao SQL. % e _ mantêm o significado de curingas de LIKE.

RESPOSTA 05

Atualizar e-mail

Arquivo completo: Exercicio05.java

Enunciado: Altere somente o e-mail, preserve o nome e trate e-mail inválido e repetido.

```
import java.sql.*;
import java.util.Scanner;

public class Exercicio05 {
    public static void main(String[] args) throws SQLException {
        try (Scanner teclado = new Scanner(System.in)) {
            System.out.print("ID: ");
            int id = Integer.parseInt(teclado.nextLine());
            System.out.print("Novo e-mail: ");
            String email = teclado.nextLine().trim();
            if (!email.contains("@") || email.contains(" ")
                    || email.length() > 150) {
                System.out.println("E-mail invalido.");
                return;
            }
            String sql = "UPDATE alunos SET email = ? WHERE id = ?";
            try (Connection c = Conexao.conectar();
                 PreparedStatement ps = c.prepareStatement(sql)) {
                ps.setString(1, email);
                ps.setInt(2, id);
                int linhas = ps.executeUpdate();
                System.out.println(linhas > 0 ? "Atualizado!" : "ID inexistente.");
            } catch (SQLException e) {
                if (e.getErrorCode() != 1062) throw e;
                System.out.println("E-mail ja cadastrado.");
            }
        }
    }
}
```

Como conferir: Digite um ID existente e outro e-mail fictício. Teste sem @ e com um e-mail já cadastrado. O UPDATE altera apenas a coluna email.

RESPOSTA 06

Exclusão segura

Arquivo completo: Exercicio06.java

Enunciado: Mostre os dados, peça S/N e trate o ID ausente.

```
import java.sql.*;
import java.util.Scanner;

public class Exercicio06 {
    public static void main(String[] args) throws SQLException {
        try (Scanner teclado = new Scanner(System.in);
             Connection c = Conexao.conectar();
             PreparedStatement buscar = c.prepareStatement(
                 "SELECT nome, email FROM alunos WHERE id = ?");
             PreparedStatement excluir = c.prepareStatement(
                 "DELETE FROM alunos WHERE id = ?")) {
            System.out.print("ID: ");
            int id = Integer.parseInt(teclado.nextLine());
            buscar.setInt(1, id);
            try (ResultSet rs = buscar.executeQuery()) {
                if (!rs.next()) {
                    System.out.println("Aluno nao encontrado.");
                    return;
                }
                System.out.println(rs.getString("nome") + " - "
                    + rs.getString("email"));
            }
            System.out.print("Excluir? S/N: ");
            if (!teclado.nextLine().equalsIgnoreCase("S")) {
                System.out.println("Cancelado.");
                return;
            }
            excluir.setInt(1, id);
            System.out.println("Excluidos: " + excluir.executeUpdate());
        }
    }
}
```

Como conferir: Com N, o registro permanece. Com S, ele é excluído. Com 999999, aparece a mensagem de aluno não encontrado.

RESPOSTA 07

Contar alunos

Arquivo completo: Exercicio07.java

Enunciado: Use COUNT(*) e compare antes/depois de um cadastro e em banco de testes vazio.

```
import java.sql.*;

public class Exercicio07 {
    public static void main(String[] args) throws SQLException {
        try (Connection c = Conexao.conectar();
             PreparedStatement ps = c.prepareStatement(
                 "SELECT COUNT(*) AS total FROM alunos");
             ResultSet rs = ps.executeQuery()) {
            rs.next();
            System.out.println("Total: " + rs.getInt("total"));
        }
    }
}
```

Como conferir: No banco novo, o total começa em zero. Depois de executar 02 pela primeira vez, deve ser três. Se você já alterou a base, o resultado corresponde aos registros atuais.

RESPOSTA 08 / PARTE 1 DE 2

Adicionar turma

Arquivo completo: Exercicio08.java

Enunciado: Adicione turma VARCHAR(20) sem perder alunos e adapte cadastrar/listar/alterar.

```
import java.sql.*;
import java.util.Scanner;

public class Exercicio08 {
    public static void main(String[] args) throws SQLException {
        try (Scanner teclado = new Scanner(System.in);
             Connection c = Conexao.conectar()) {
            System.out.println("1 Cadastrar | 2 Listar | 3 Alterar");
            String opcao = teclado.nextLine();
            if (opcao.equals("2")) {
                try (PreparedStatement ps = c.prepareStatement(
                        "SELECT id, nome, email, turma FROM alunos ORDER BY id");
                     ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        System.out.println(rs.getInt("id") + " | "
                            + rs.getString("nome") + " | "
                            + rs.getString("email") + " | "
                            + rs.getString("turma"));
                    }
                }
                return;
            }
            if (!opcao.equals("1") && !opcao.equals("3")) {
                System.out.println("Opcao invalida.");
                return;
            }
            int id = 0;
            if (opcao.equals("3")) {
```

RESPOSTA 08 / PARTE 2 DE 2

Adicionar turma

Arquivo completo: Exercicio08.java • continuação do mesmo arquivo

Continue o código exatamente depois da última linha da página anterior.

```
                System.out.print("ID: ");
                id = Integer.parseInt(teclado.nextLine());
            }
            System.out.print("Nome: ");
            String nome = teclado.nextLine().trim();
            System.out.print("E-mail: ");
            String email = teclado.nextLine().trim();
            System.out.print("Turma: ");
            String turma = teclado.nextLine().trim();
            if (nome.isEmpty() || nome.length() > 100
                    || !email.contains("@") || email.length() > 150
                    || turma.isEmpty() || turma.length() > 20) {
                System.out.println("Dados invalidos. Turma: ate 20 caracteres.");
                return;
            }
            String sql = opcao.equals("1")
                ? "INSERT INTO alunos (nome, email, turma) VALUES (?, ?, ?)"
                : "UPDATE alunos SET nome = ?, email = ?, turma = ? WHERE id = ?";
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                ps.setString(1, nome);
                ps.setString(2, email);
                ps.setString(3, turma);
                if (opcao.equals("3")) ps.setInt(4, id);
                System.out.println("Registros gravados: " + ps.executeUpdate());
            }
        }
    }
}
```

Como conferir: Execute o SQL abaixo uma vez como root. O programa tem as opções 1 cadastrar, 2 listar e 3 alterar. Até 20 caracteres são aceitos. Nos alunos antigos, turma fica NULL até ser preenchida.

APOIO / EXERCÍCIO 08

Prepare e confira

SQL complementar ou roteiro de teste da resposta anterior.

```sql
-- Execute como administrador, uma vez, antes do exercicio 8.
-- Preserva os alunos existentes: turma fica NULL nos registros antigos.
USE escola_maximus;
ALTER TABLE alunos ADD COLUMN turma VARCHAR(20);
```

Execute como administrador **uma vez**, antes de rodar Exercicio08. Se aparecer Duplicate column name, a coluna já existe e o comando não precisa ser repetido.

Os registros antigos permanecem no banco e recebem NULL em turma. Use a opção 3 para preencher os dados. A opção 2 lista os registros; a opção 1 cadastra um novo aluno.

> Teste turma com até 20 caracteres e outra com 21. A segunda deve ser recusada. Use um e-mail diferente a cada novo cadastro, pois a tabela proíbe e-mails repetidos.

RESPOSTA 09 / PARTE 1 DE 2

Validações e erros

Arquivo completo: Exercicio09.java

Enunciado: Rejeite nome com um caractere, e-mail vazio e ID negativo; teste cinco erros mantendo o menu aberto.

```
import java.sql.*;
import java.util.Scanner;

public class Exercicio09 {
    public static void main(String[] args) {
        try (Scanner teclado = new Scanner(System.in)) {
            while (true) {
                System.out.println("1 Cadastrar | 2 Excluir | 0 Sair");
                if (!teclado.hasNextLine()) break;
                String opcao = teclado.nextLine();
                if (opcao.equals("0")) break;
                try (Connection c = Conexao.conectar()) {
                    if (opcao.equals("1")) {
                        System.out.print("Nome: ");
                        String nome = teclado.nextLine().trim();
                        System.out.print("E-mail: ");
                        String email = teclado.nextLine().trim();
                        if (nome.length() < 2 || nome.length() > 100
                                || !email.contains("@") || email.contains(" ")
                                || email.length() > 150) {
                            System.out.println("Nome ou e-mail invalido.");
                            continue;
                        }
                        try (PreparedStatement ps = c.prepareStatement(
                                "INSERT INTO alunos (nome, email) VALUES (?, ?)")) {
                            ps.setString(1, nome);
                            ps.setString(2, email);
                            ps.executeUpdate();
```

RESPOSTA 09 / PARTE 2 DE 2

Validações e erros

Arquivo completo: Exercicio09.java • continuação do mesmo arquivo

Continue o código exatamente depois da última linha da página anterior.

```
                        System.out.println("Cadastrado!");
                    }
                } else if (opcao.equals("2")) {
                    System.out.print("ID: ");
                    int id = Integer.parseInt(teclado.nextLine());
                    if (id <= 0) {
                        System.out.println("ID deve ser positivo.");
                        continue;
                    }
                    System.out.print("Excluir? S/N: ");
                    if (!teclado.nextLine().equalsIgnoreCase("S")) continue;
                    try (PreparedStatement ps = c.prepareStatement(
                            "DELETE FROM alunos WHERE id = ?")) {
                        ps.setInt(1, id);
                        System.out.println("Excluidos: " + ps.executeUpdate());
                    }
                } else {
                    System.out.println("Opcao invalida.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Digite um numero inteiro.");
            } catch (SQLException e) {
                System.out.println("Erro no banco: " + e.getMessage());
            }
        }
    }
}
```

Como conferir: O menu usa while e continue. A tabela após o código mostra cinco casos de teste. A entrada correta pode ser cadastrada logo depois de um erro.

APOIO / EXERCÍCIO 09

Prepare e confira

SQL complementar ou roteiro de teste da resposta anterior.

01 Nome curto
1 → A → a@example.com. O programa deve mostrar a mensagem correspondente e retornar ao menu.

02 E-mail vazio
1 → Ana → linha vazia. O programa deve mostrar a mensagem correspondente e retornar ao menu.

03 ID negativo
2 → -1. O programa deve mostrar a mensagem correspondente e retornar ao menu.

04 ID com texto
2 → abc. O programa deve mostrar a mensagem correspondente e retornar ao menu.

05 E-mail duplicado
1 → Outro Nome → e-mail já cadastrado. O programa deve mostrar a mensagem correspondente e retornar ao menu.

Depois dos erros, cadastre um nome com pelo menos dois caracteres e um e-mail fictício novo. O cadastro deve funcionar. Digite 0 para sair.

> A validação de e-mail é básica: exige @, impede espaços e respeita o limite de tamanho. Não verifica se o endereço existe nem todas as regras de formato.

RESPOSTA 10 / PARTE 1 DE 3

Mini-sistema de produtos

Arquivo completo: Exercicio10.java

Enunciado: Faça CRUD de produtos com BigDecimal, preço e estoque não negativos e confirmação de exclusão.

```
import java.math.BigDecimal;
import java.sql.*;
import java.util.Scanner;

public class Exercicio10 {
    public static void main(String[] args) throws SQLException {
        try (Scanner teclado = new Scanner(System.in);
             Connection c = Conexao.conectar()) {
            System.out.println("1 Cadastrar | 2 Listar | 3 Alterar | 4 Excluir");
            String opcao = teclado.nextLine();
            if (opcao.equals("2")) {
                try (PreparedStatement ps = c.prepareStatement(
                        "SELECT id, nome, preco, estoque FROM produtos ORDER BY id");
                     ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        System.out.println(rs.getInt("id") + " | "
                            + rs.getString("nome") + " | R$ "
                            + rs.getBigDecimal("preco") + " | "
                            + rs.getInt("estoque"));
                    }
                }
                return;
            }
            if (!opcao.equals("1") && !opcao.equals("3") && !opcao.equals("4")) {
                System.out.println("Opcao invalida.");
```

RESPOSTA 10 / PARTE 2 DE 3

Mini-sistema de produtos

Arquivo completo: Exercicio10.java • continuação do mesmo arquivo

Continue o código exatamente depois da última linha da página anterior.

```
            return;
        }
        int id = 0;
        if (!opcao.equals("1")) {
            System.out.print("ID: ");
            id = Integer.parseInt(teclado.nextLine());
            if (id <= 0) {
                System.out.println("ID invalido.");
                return;
            }
        }
        if (opcao.equals("4")) {
            System.out.print("Excluir? S/N: ");
            if (!teclado.nextLine().equalsIgnoreCase("S")) return;
            try (PreparedStatement ps = c.prepareStatement(
                    "DELETE FROM produtos WHERE id = ?")) {
                ps.setInt(1, id);
                int linhas = ps.executeUpdate();
                System.out.println(linhas > 0 ? "Excluido!" : "ID inexistente.");
            }
            return;
        }
        System.out.print("Nome: ");
        String nome = teclado.nextLine().trim();
        System.out.print("Preco (ex.: 12.50): ");
```

RESPOSTA 10 / PARTE 3 DE 3

Mini-sistema de produtos

Arquivo completo: Exercicio10.java • continuação do mesmo arquivo

Continue o código exatamente depois da última linha da página anterior.

```
            BigDecimal preco = new BigDecimal(teclado.nextLine().replace(",", "."));
            System.out.print("Estoque: ");
            int estoque = Integer.parseInt(teclado.nextLine());
            if (nome.isEmpty() || nome.length() > 100 || preco.signum() < 0
                    || preco.compareTo(new BigDecimal("99999999.99")) > 0
                    || preco.stripTrailingZeros().scale() > 2 || estoque < 0) {
                System.out.println("Dados invalidos; preco: ate 2 casas decimais.");
                return;
            }
            String sql = opcao.equals("1")
                ? "INSERT INTO produtos (nome, preco, estoque) VALUES (?, ?, ?)"
                : "UPDATE produtos SET nome = ?, preco = ?, estoque = ? WHERE id = ?";
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                ps.setString(1, nome);
                ps.setBigDecimal(2, preco);
                ps.setInt(3, estoque);
                if (opcao.equals("3")) ps.setInt(4, id);
                int linhas = ps.executeUpdate();
                System.out.println(linhas > 0 ? "Gravado!" : "ID inexistente.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Numero invalido.");
        }
    }
}
```

Como conferir: Execute o SQL abaixo como root. Rode a classe para cadastrar, listar, alterar ou excluir. Teste preço negativo, estoque negativo e ID 999999 nas opções de alterar/excluir.

APOIO / EXERCÍCIO 10

Prepare e confira

SQL complementar ou roteiro de teste da resposta anterior.

```sql
-- Execute como administrador antes do exercicio 10.
USE escola_maximus;
CREATE TABLE IF NOT EXISTS produtos (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  preco DECIMAL(10,2) NOT NULL CHECK (preco >= 0),
  estoque INT NOT NULL CHECK (estoque >= 0)
);
```

Execute como administrador antes de Exercicio10. O usuário da aula já tem permissão de dados para as tabelas deste banco. BigDecimal mantém o valor decimal do preço; não usamos double para dinheiro.

Rode a opção 1 com Caderno, 12.50 e estoque 3. Liste com 2 e anote o ID. Altere com 3. Para excluir, use 4 e confirme S. Com N, o produto permanece.

> Teste preço negativo, estoque negativo, preço com três casas decimais e o ID 999999. O programa deve rejeitar valores inválidos e informar quando o ID não existe.

FINAL / CONSULTA

Dúvidas mais comuns

Confira a mensagem do Console e volte à etapa indicada.

No suitable driver
Confira Maven Dependencies ou o JAR no Classpath.

Access denied
Confira usuário, senha e o script 02. Usuário já existente pode ter outra senha.

Unknown database
Execute sql/01_banco.sql e confira o nome do banco.

Coluna turma ou tabela produtos ausente
Execute o SQL complementar do exercício 08 ou 10 como administrador.

Communications link failure
Confira se o serviço MySQL está ligado e a porta 3306 está correta.

NumberFormatException
Digite número nos campos numéricos. O exercício 09 mostra como tratar esse erro.

README com as 10 respostas completas
https://github.com/perabru/java-mysql#respostas-dos-10-exercícios

Verificações automáticas no MySQL
https://github.com/perabru/java-mysql/actions

Referências oficiais: documentação JDBC do Java 21 e guia do MySQL Connector/J. Links de download na página 2. Logo e paleta: site da Escola Máximu's. Revisão: 20/10/2026.